zeroriscy_int_controller

OVERVIEW

The zeroriscy_int_controller module is responsible for handling external interrupt requests in the Zero-Riscy RISC-V core. It detects interrupt signals, manages communication with the CPU control logic, and ensures proper interrupt servicing using a Finite State Machine (FSM).

This module acts as an interface between:

- External interrupt sources
- CPU control and execution logic

Its main purpose is to guarantee that interrupts are handled safely, efficiently, and in the correct sequence without disturbing normal processor operation.

The interrupt controller manages interrupt detection, interrupt acknowledgment, cancellation handling, and interrupt completion using a simple and reliable control mechanism.

KEY FEATURES

- Supports external interrupt handling
- Implements a 3-state FSM for interrupt flow control
- Includes handshake mechanism with the controller
- Global interrupt enable control using m_IE_i
- Stores and forwards interrupt ID
- Supports interrupt acknowledgment and cancellation
- Fully synchronous design with asynchronous reset

FUNCTIONAL DESCRIPTION

The interrupt controller continuously monitors incoming interrupt requests and decides whether they should be serviced.

Its working process includes:

- Monitoring the external interrupt signal irq_i
- Checking whether global interrupts are enabled using m_IE_i
- Capturing the interrupt ID from irq_id_i
- Sending an interrupt request to the control unit using irq_req_ctrl_o
- Waiting for either: Acknowledgment (ctrl_ack_i) -> continue servicing, or Kill signal (ctrl_kill_i) -> cancel interrupt request
- Returning to idle state after completion

This structured flow ensures safe and predictable interrupt handling.

INPUT SIGNALS

- clk [1-bit]: System clock
- rst_n [1-bit]: Active-low reset
- ctrl_ack_i [1-bit]: Acknowledge signal from control unit
- ctrl_kill_i [1-bit]: Cancel interrupt request
- irq_i [1-bit]: External interrupt signal
- irq_id_i [5-bit]: Interrupt ID from external source
- m_IE_i [1-bit]: Global interrupt enable

OUTPUT AND INTERNAL SIGNALS

Outputs:

- irq_req_ctrl_o [1-bit]: Interrupt request sent to controller
- irq_id_ctrl_o [5-bit]: Latched interrupt ID

Internal Signals:

- exc_ctrl_cs: Current FSM state
- irq_id_q: Stored interrupt ID
- irq_enable_ext: Interrupt enable condition (m_IE_i)

These internal signals help manage interrupt state transitions and maintain correct interrupt tracking.

FINITE STATE MACHINE (FSM)

The interrupt controller uses a 3-state FSM to manage interrupt servicing.

FSM States:

- IDLE (00): Waiting for interrupt
- IRQ_PENDING (01): Interrupt request is active
- IRQ_DONE (10): Interrupt has been serviced

This FSM ensures orderly interrupt handling and prevents invalid state transitions.

FSM OPERATION: 1. IDLE STATE

This is the default state where the controller waits for a valid interrupt request.

Condition to Enter Next State: irq_i = 1 AND m_IE_i = 1

Actions Performed:

- Capture the interrupt ID from irq_id_i
- Store it in irq_id_q
- Move to IRQ_PENDING

If interrupts are disabled, the request is ignored.

FSM OPERATION: 2. IRQ_PENDING STATE

In this state, the interrupt request becomes active.
Output Behavior: irq_req_ctrl_o = 1. This notifies the control unit that an interrupt needs servicing.
Wait Conditions:

- Acknowledgment (ctrl_ack_i = 1) -> Move to IRQ_DONE
- Kill Signal (ctrl_kill_i = 1) -> Return to IDLE
- Otherwise -> Remain in IRQ_PENDING

This ensures the interrupt is either serviced or safely canceled.

FSM OPERATION: 3. IRQ_DONE STATE

This state indicates that interrupt processing has been completed.
Action:

- Interrupt lifecycle is completed
- Controller automatically returns to IDLE

This prepares the system for the next interrupt request.

OUTPUT AND TIMING BEHAVIOR

Output Behavior:

- irq_req_ctrl_o: This signal becomes active only during IRQ_PENDING. This ensures interrupt requests are asserted only when required.
- irq_id_ctrl_o: This output always reflects the stored interrupt ID (irq_id_q), guaranteeing correct tracking.

Timing Behavior:
All state transitions occur on posedge clk. This makes the design fully synchronous and predictable.

Reset Behavior:
When rst_n = 0, the controller initializes the FSM state to IDLE and the Interrupt ID to 0. This ensures safe startup and reliable recovery after reset.

EDGE CASE HANDLING

- Interrupt when disabled -> Ignored
- Kill signal during pending state -> Interrupt request is canceled
- Multiple simultaneous interrupts -> Only one handled at a time
- Reset asserted -> Clears FSM state and stored ID

This improves system stability and prevents undefined behavior.

DESIGN ADVANTAGES & SUMMARY

Design Advantages:

- Simple and efficient FSM-based design
- Reliable handshake mechanism with control unit
- Safe and deterministic interrupt handling
- Easy integration with pipelined CPUs

- Low hardware complexity
- Clean reset and cancellation behavior

SUMMARY:
The zeroriscy_int_controller is responsible for reliable external interrupt management in the Zero-Riscy processor. It detects valid interrupt requests, verifies global interrupt enable conditions, coordinates interrupt servicing with CPU control logic, and uses FSM-based control for structured interrupt flow. Its simple architecture, deterministic behavior, and low hardware overhead make it an efficient and essential module for stable interrupt handling in compact RISC-V processor designs.